

Section A: Description of Database Application area and ER Model

1. Application Description.....
2. Entity Relationship Diagram.....
3. Mapping to Relational Schema
4. Functional Dependency Diagrams (for proposed relations)

Section B Explanation of data and SQL Code:

5. Explanation of the SQL Code for Creating one of your database Tables (including any constraints).....
6. Explanation and SQL Code for any Altering tables.....
7. Explanation and SQL Code for any Trigger operations.....
8. Explanation and SQL Code for and Creation of Views.....
9. Explanation and SQL Code for one of your commands to Populate a Tables.....
10. Explanation and example SQL Code for retrieving information from the database (including any use of Joins and use of functions).....
11. Explanation and SQL Code for any Security commands (roles & permissions).....
12. Explanation of and Additional SQL Features of your choice

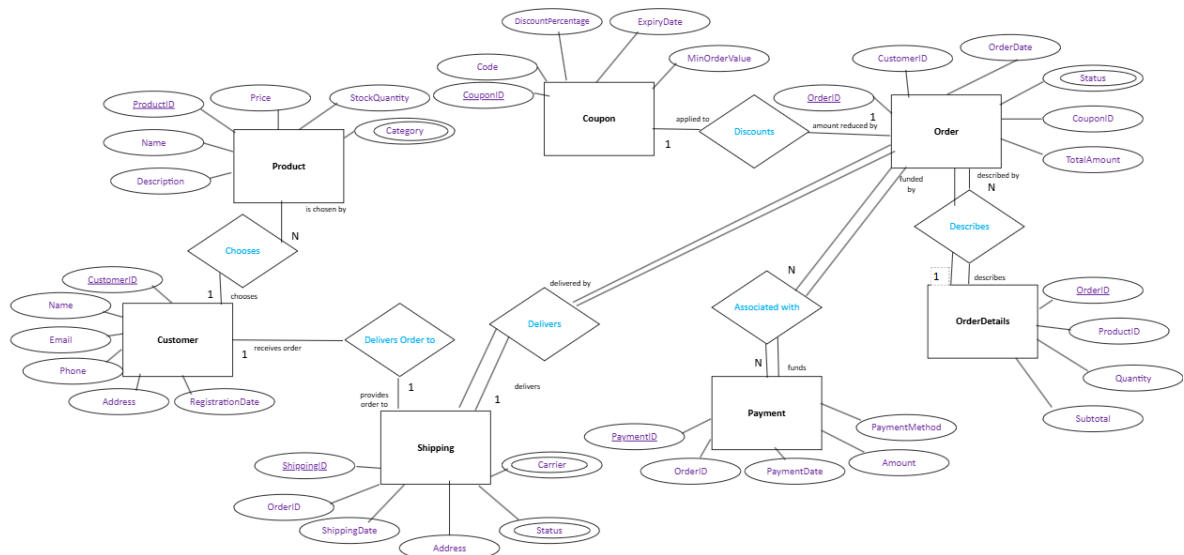
Section C Listing of your SQL Code for the database

Section A:

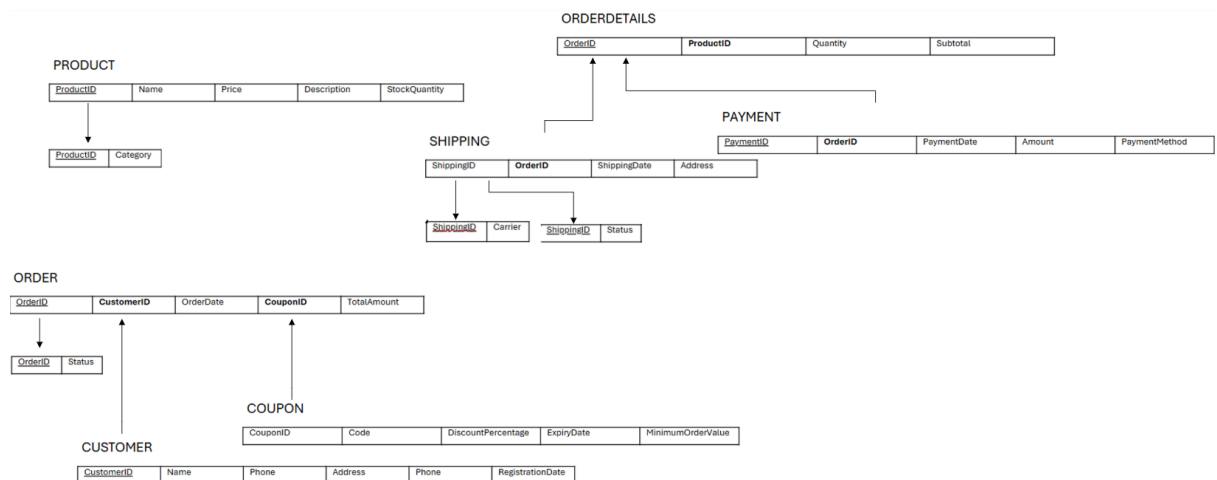
1. *Application Description*

The application that I have chosen to create my MySQL database for is an e-commerce platform for electronics. I chose this topic as e-commerce has become extremely prevalent in recent years with the rise in online shopping. I also understood that there would be many features that could make good use of MySQL functionalities; such as implementing coupons for discounts and different methods of shipping and payment.

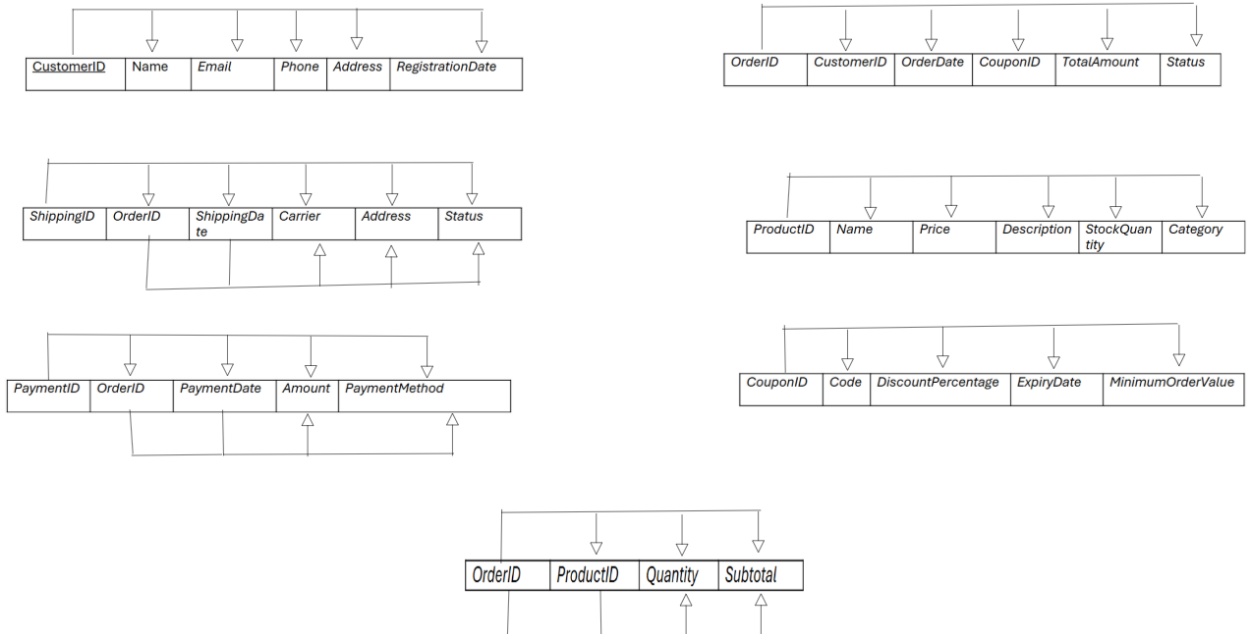
2. Entity Relationship Diagram



3. Mapping to Relational Schema



4. Functional Dependency Diagrams



Section B

5. Explanation of the SQL Code for Creating one of your database Tables

Table structure									
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	CustomerID	int(11)			No	None	Unique ID for customers	AUTO_INCREMENT	Change Drop More
☐ 2	Name	varchar(100)	utf8mb3_general_ci		No	None	Customer full name		Change Drop More
☐ 3	Email	varchar(100)	utf8mb3_general_ci		No	None	Customer email		Change Drop More
☐ 4	Phone	varchar(20)	utf8mb3_general_ci		Yes	NULL	Customer phone number		Change Drop More
☐ 5	Address	text	utf8mb3_general_ci		No	None	Customer address		Change Drop More
☐ 6	RegistrationDate	datetime			No	current_timestamp()	Date when customer registered		Change Drop More

```

CREATE TABLE `Customer` (
  `CustomerID` int(11) NOT NULL COMMENT 'Unique ID for customers',
  `Name` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT 'Customer full name',
  `Email` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT 'Customer email',
  `Phone` varchar(20) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci DEFAULT NULL COMMENT 'Customer phone number',
  `Address` text CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL COMMENT 'Customer address',
  `RegistrationDate` datetime NOT NULL DEFAULT current_timestamp() COMMENT 'Date when customer registered'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `Customer` (`CustomerID`, `Name`, `Email`, `Phone`, `Address`, `RegistrationDate`) VALUES
(1, 'Hayley Williams', 'hayley@gmail.com', '0831111111', '123 Elm St', '2025-02-01 00:00:00'),
(2, 'Taylor York', 'taylor@yahoo.com', '0852222222', '456 Oak St', '2025-01-15 00:00:00'),
(3, 'Brian OConnor', 'brian@hotmail.com', '0863333333', '789 Pine St', '2025-01-30 00:00:00'),
(4, 'Zac Farro', 'zac@hotmail.co.uk', '0874444444', '101 Maple St', '2025-02-10 00:00:00'),
(5, 'Chad Gilbert', 'chad@icloud.com', '0895555555', '202 Birch St', '2025-01-25 00:00:00');

ALTER TABLE `Customer`
  ADD PRIMARY KEY (`CustomerID`),
  ADD UNIQUE KEY `Email` (`Email`);

ALTER TABLE `Customer`
  MODIFY `CustomerID` int(11) NOT NULL AUTO_INCREMENT COMMENT 'Unique ID for customers', AUTO_INCREMENT=6;
COMMIT;

```

This is the Customer table in my database. Each customer consists of six attributes:

- Their unique customer ID
- Their name
- Their email
- Their phone number
- Their address
- The date they registered with the e-commerce platform

The customer ID is considered the primary key for this table and the email must also be unique.

The customer ID has an auto-increment feature which is executed when a new customer is added.

All fields except Phone are defaulted to NOT NULL.

Five sample customers have been added into the database.

6. Explanation and SQL Code for any Altering tables

	OrderID	CustomerID	OrderDate	Status	CouponID	TotalAmount
	Unique order ID	Links to Customer table	Date and time of order	Current order status	Links to Coupon table	
☐ Edit ☐ Copy ☐ Delete	1	1	2025-03-01 00:00:00	Pending	NULL	3200.00
☐ Edit ☐ Copy ☐ Delete	2	2	2025-03-02 00:00:00	Pending	4	475.00
☐ Edit ☐ Copy ☐ Delete	3	3	2025-03-02 00:00:00	Shipped	NULL	1350.00
☐ Edit ☐ Copy ☐ Delete	4	4	2025-03-03 00:00:00	Delivered	NULL	800.00
☐ Edit ☐ Copy ☐ Delete	5	5	2025-03-03 00:00:00	Cancelled	NULL	0.00

```
ALTER TABLE `Orders`
  ADD PRIMARY KEY (`OrderID`),
  ADD KEY `CustomerID` (`CustomerID`),
  ADD KEY `CouponID` (`CouponID`);

ALTER TABLE `Orders`
  MODIFY `OrderID` int(11) NOT NULL AUTO_INCREMENT COMMENT 'Unique order ID', AUTO_INCREMENT=6;

ALTER TABLE `Orders`
  ADD CONSTRAINT `Orders_ibfk_1` FOREIGN KEY (`CustomerID`) REFERENCES `Customer` (`CustomerID`) ON DELETE CASCADE ON UPDATE CASCADE,
  ADD CONSTRAINT `Orders_ibfk_2` FOREIGN KEY (`CouponID`) REFERENCES `Coupon` (`CouponID`) ON DELETE SET NULL ON UPDATE CASCADE;
COMMIT;
```

My Orders table is an example of a table with several alters.

The first ALTER statement defines the OrderID as the primary key of the table. It also defines CustomerID and CouponID as foreign keys which link to the Customer and Coupon tables respectively.

The second ALTER statement ensures that the OrderID field cannot be null and that when new orders are added to the table, the OrderID of that entry will auto-increment.

The final ALTER statement defines constraints on the table. It ensures that if a customer is deleted from the database, all orders that are related to that customer will also be deleted. Additionally, if the CouponID in the Coupon table is updated, the corresponding CouponID in the Orders table will also be updated automatically.

7. Explanation and SQL Code for any Trigger operations

	Name	Table	Time	Event			
☐	assign_default_role	Customer	BEFORE	INSERT			
☐	reduce_stock_on_order	OrderDetails	AFTER	INSERT			
☐	restore_stock_on_order_cancel	OrderDetails	AFTER	DELETE			

```

1 CREATE TRIGGER `assign_default_role` BEFORE INSERT ON `Customer`
2   FOR EACH ROW BEGIN
3     IF NEW.role_id IS NULL THEN
4       SET NEW.role_id = (SELECT role_id FROM Roles WHERE role_name = 'Customer' LIMIT 1);
5     END IF;
6   END
7
8 CREATE TRIGGER `reduce_stock_on_order` AFTER INSERT ON `OrderDetails`
9   FOR EACH ROW BEGIN
10    UPDATE Product
11    SET stock_quantity = stock_quantity - NEW.quantity
12    WHERE product_id = NEW.productID;
13  END
14
15 CREATE TRIGGER `restore_stock_on_order_cancel` AFTER DELETE ON `OrderDetails`
16   FOR EACH ROW BEGIN
17    UPDATE Product
18    SET stock_quantity = stock_quantity + OLD.quantity
19    WHERE product_id = OLD.productid;
20  END

```

An example of 3 trigger operations in my database, related to the OrderDetails and Customer tables.

- A. The first trigger ensures that each user that gains access to the database is automatically granted the role of “Customer” and not the “Admin” role.
- B. The second trigger automatically decrements a product’s available stock quantity field when the product is included in an order. If the product appears

multiple times in an order (e.g. a customer orders 3 laptops, the quantity in stock for the laptop product will decrease by 3)

- C. The final trigger will check to see if an order is cancelled. If this happens, it will restore the quantity number of the product. (e.g. if a customer orders a mobile phone and cancels the order, the quantity in stock for the mobile phone product will increment by 1).

8. Explanation and SQL Code for and Creation of Views

Table	Action	Rows	Type	Collation	Size	Overhead
☐ ActiveOrders	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ CouponUsageSummary	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ CustomerOrdersSummary	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ OrderDetailsWithProducts	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ OrderPaymentAndShippingStatus	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ OrdersWithCoupons	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ OrdersWithPayments	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ OrdersWithShipping	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ ProductInventorySummary	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-
☐ TotalSalesPerCustomer	★ Browse Structure Search Insert Edit Drop	~0	View	---	-	-

```

1 CREATE TABLE `ActiveOrders` (
2   `OrderID` int(11)
3   , `OrderDate` datetime
4   , `Status` enum('Pending','Shipped','Delivered','Cancelled')
5   , `CustomerName` varchar(100)
6 );
7
8 CREATE TABLE `CouponUsageSummary` (
9   `OrderID` int(11)
10  , `CouponCode` varchar(20)
11  , `OrderDate` datetime
12  , `OrderStatus` enum('Pending','Shipped','Delivered','Cancelled')
13  , `CouponID` int(11)
14  , `DiscountPercentage` decimal(5,2)
15 );
16
17 CREATE TABLE `CustomerOrdersSummary` (
18   `CustomerID` int(11)
19   , `CustomerName` varchar(100)
20   , `OrderID` int(11)
21   , `OrderDate` datetime
22   , `OrderStatus` enum('Pending','Shipped','Delivered','Cancelled')
23   , `CouponCode` varchar(20)
24   , `TotalOrderAmount` decimal(32,2)
25 );
26
27 CREATE TABLE `OrderDetailsWithProducts` (
28   `OrderID` int(11)
29   , `ProductID` int(11)
30   , `ProductName` varchar(100)
31   , `Quantity` int(11)
32   , `Subtotal` decimal(10,2)
33 );
34
35 CREATE TABLE `OrderPaymentAndShippingStatus` (
36   `OrderID` int(11)
37   , `OrderDate` datetime
38   , `OrderStatus` enum('Pending','Shipped','Delivered','Cancelled')
39   , `PaymentMethod` enum('Credit Card','PayPal','Bank Transfer')
40   , `PaymentDate` datetime
41   , `PaidAmount` decimal(10,0)
42   , `ShippingDate` datetime
43   , `ShippingStatus` enum('Processing','Shipping in Progress','Delivered','')
44 );
45
46 CREATE TABLE `OrdersWithCoupons` (
47   `OrderID` int(11)
48   , `CustomerName` varchar(100)
49   , `OrderDate` datetime
50   , `Status` enum('Pending','Shipped','Delivered','Cancelled')
51   , `CouponCode` varchar(20)
52   , `DiscountPercentage` decimal(5,2)
53 );

```

```

55 CREATE TABLE `OrdersWithPayments` (
56   `OrderID` int(11)
57   , `OrderDate` datetime
58   , `OrderStatus` enum('Pending','Shipped','Delivered','Cancelled')
59   , `PaymentID` int(11)
60   , `PaymentDate` datetime
61   , `PaidAmount` decimal(10,0)
62   , `PaymentMethod` enum('Credit Card','PayPal','Bank Transfer')
63 );
64
65 CREATE TABLE `OrdersWithShipping` (
66   `OrderID` int(11)
67   , `OrderDate` datetime
68   , `OrderStatus` enum('Pending','Shipped','Delivered','Cancelled')
69   , `ShippingID` int(11)
70   , `ShippingDate` datetime
71   , `ShippingStatus` enum('Processing','Shipping in Progress','Delivered','')
72   , `Carrier` varchar(50)
73 );
74
75 CREATE TABLE `ProductInventorySummary` (
76   `ProductID` int(11)
77   , `ProductName` varchar(100)
78   , `Price` decimal(10,0)
79   , `StockQuantity` int(11)
80   , `TotalSold` decimal(32,0)
81 );
82
83 CREATE TABLE `TotalSalesPerCustomer` (
84   `CustomerID` int(11)
85   , `CustomerName` varchar(100)
86   , `TotalSpent` decimal(32,2)
87 );

```

There are 10 views in my database.

Above is the corresponding tables to the views. Below is the code for the views themselves:

```
-- Structure for view 'ActiveOrders'
DROP TABLE IF EXISTS 'ActiveOrders';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'ActiveOrders' AS SELECT 'O'.'OrderID' AS 'OrderID', 'O'.'OrderDate' AS 'OrderDate', 'O'.'Status' AS 'Status', 'C'.'Name' AS 'CustomerName'
FROM ('Orders' 'O' JOIN 'Customer' 'C' ON('O'.'CustomerID' = 'C'.'CustomerID')) WHERE 'O'.'Status' IN ('Pending','Shipped','Delivered');

-- Structure for view 'CouponUsageSummary'
DROP TABLE IF EXISTS 'CouponUsageSummary';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'CouponUsageSummary' AS SELECT 'O'.'OrderID' AS 'OrderID', 'C'.'Code' AS 'CouponCode', 'O'.'OrderDate' AS 'OrderDate', 'O'.'Status' AS 'OrderStatus', 'O'.'CouponID' AS 'CouponID', 'CP'.'DiscountPercentage' AS 'DiscountPercentage'
FROM (('Orders' 'O' LEFT JOIN 'Coupon' 'C' ON('O'.'CouponID' = 'C'.'CouponID')) LEFT JOIN 'Coupon' 'CP' ON('O'.'CouponID' = 'CP'.'CouponID')) WHERE 'O'.'CouponID' IS NOT NULL;

-- Structure for view 'CustomerOrdersSummary'
DROP TABLE IF EXISTS 'CustomerOrdersSummary';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'CustomerOrdersSummary' AS SELECT 'C'.'CustomerID' AS 'CustomerID', 'C'.'Name' AS 'CustomerName', 'O'.'OrderID' AS 'OrderID', 'O'.'OrderDate' AS 'OrderDate', 'O'.'Status' AS 'OrderStatus', 'CP'.'Code' AS 'CouponCode', sum('O'.'Subtotal') AS 'TotalOrderAmount'
FROM (('Orders' 'O' JOIN 'Customer' 'C' ON('O'.'CustomerID' = 'C'.'CustomerID')) LEFT JOIN 'Coupon' 'CP' ON('O'.'CouponID' = 'CP'.'CouponID')) JOIN 'OrderDetails' 'OD' ON('O'.'OrderID' = 'OD'.'OrderID') GROUP BY 'C'.'CustomerID', 'O'.'OrderID';

-- Structure for view 'OrderDetailsWithProducts'
DROP TABLE IF EXISTS 'OrderDetailsWithProducts';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'OrderDetailsWithProducts' AS SELECT 'OD'.'OrderID' AS 'OrderID', 'OD'.'ProductID' AS 'ProductID', 'P'.'Name' AS 'ProductName', 'OD'.'Quantity' AS 'Quantity', 'OD'.'Subtotal' AS 'Subtotal'
FROM ('OrderDetails' 'OD' JOIN 'Product' 'P' ON('OD'.'ProductID' = 'P'.'ProductID'));

-- Structure for view 'OrderPaymentAndShippingStatus'
DROP TABLE IF EXISTS 'OrderPaymentAndShippingStatus';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'OrderPaymentAndShippingStatus' AS SELECT 'O'.'OrderID' AS 'OrderID', 'O'.'OrderDate' AS 'OrderDate', 'O'.'Status' AS 'OrderStatus', 'P'.'PaymentMethod' AS 'PaymentMethod', 'P'.'PaymentDate' AS 'PaymentDate', 'P'.'Amount' AS 'PaidAmount', 'S'.'ShippingDate' AS 'ShippingDate', 'S'.'Status' AS 'ShippingStatus' FROM (('Orders' 'O' LEFT JOIN 'Payment' 'P' ON('O'.'OrderID' = 'P'.'OrderID')) LEFT JOIN 'Shipping' 'S' ON('O'.'OrderID' = 'S'.'OrderID'));

-- Structure for view 'OrdersWithCoupons'
DROP TABLE IF EXISTS 'OrdersWithCoupons';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'OrdersWithCoupons' AS SELECT 'O'.'OrderID' AS 'OrderID', 'C'.'Name' AS 'CustomerName', 'O'.'OrderDate' AS 'OrderDate', 'O'.'Status' AS 'Status', 'CP'.'Code' AS 'CouponCode', 'CP'.'DiscountPercentage' AS 'DiscountPercentage'
FROM (('Orders' 'O' LEFT JOIN 'Customer' 'C' ON('O'.'CustomerID' = 'C'.'CustomerID')) LEFT JOIN 'Coupon' 'CP' ON('O'.'CouponID' = 'CP'.'CouponID'));

-- Structure for view 'OrdersWithPayments'
DROP TABLE IF EXISTS 'OrdersWithPayments';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'OrdersWithPayments' AS SELECT 'O'.'OrderID' AS 'OrderID', 'O'.'OrderDate' AS 'OrderDate', 'O'.'Status' AS 'OrderStatus', 'P'.'PaymentID' AS 'PaymentID', 'P'.'PaymentDate' AS 'PaymentDate', 'P'.'Amount' AS 'PaidAmount', 'P'.'PaymentMethod' AS 'PaymentMethod'
FROM ('Orders' 'O' LEFT JOIN 'Payment' 'P' ON('O'.'OrderID' = 'P'.'OrderID'));

-- Structure for view 'OrdersWithShipping'
DROP TABLE IF EXISTS 'OrdersWithShipping';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'OrdersWithShipping' AS SELECT 'O'.'OrderID' AS 'OrderID', 'O'.'OrderDate' AS 'OrderDate', 'O'.'Status' AS 'OrderStatus', 'S'.'ShippingID' AS 'ShippingID', 'S'.'ShippingDate' AS 'ShippingDate', 'S'.'Status' AS 'ShippingStatus', 'S'.'Carrier' AS 'Carrier'
FROM ('Orders' 'O' LEFT JOIN 'Shipping' 'S' ON('O'.'OrderID' = 'S'.'OrderID'));

-- Structure for view 'ProductInventorySummary'
DROP TABLE IF EXISTS 'ProductInventorySummary';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'ProductInventorySummary' AS SELECT 'P'.'ProductID' AS 'ProductID', 'P'.'Name' AS 'ProductName', 'P'.'Price' AS 'Price', 'P'.'StockQuantity' AS 'StockQuantity', coalesce(sum('OD'.'Quantity'),0) AS 'TotalSold'
FROM ('Product' 'P' LEFT JOIN 'OrderDetails' 'OD' ON('P'.'ProductID' = 'OD'.'ProductID')) GROUP BY 'P'.'ProductID';

-- Structure for view 'TotalSalesPerCustomer'
DROP TABLE IF EXISTS 'TotalSalesPerCustomer';
CREATE ALGORITHM=UNDEFINED DEFINER=bruno@172.17.0.1 SQL SECURITY DEFINER VIEW 'TotalSalesPerCustomer' AS SELECT 'C'.'CustomerID' AS 'CustomerID', 'C'.'Name' AS 'CustomerName', sum('OD'.'Subtotal') AS 'TotalSales'
FROM (('Orders' 'O' JOIN 'Customer' 'C' ON('O'.'CustomerID' = 'C'.'CustomerID')) JOIN 'OrderDetails' 'OD' ON('O'.'OrderID' = 'OD'.'OrderID')) GROUP BY 'C'.'CustomerID';
```

Explanation of the Views:

View Name	Table	Joins to...	Purpose
ActiveOrders	Orders	Customer	Displays active orders
CouponUsage Summary	Orders	Coupon	Displays information about orders using coupons
CustomerOrders Summary	Orders	Customer/ Coupon	Summarizes customer orders
OrderDetailsWith Products	Order Details	Product	Displays order details, including OrderID, ProductID, ProductName, Quantity, and Subtotal
OrderPaymentAnd ShippingStatus	Orders	Payment/ Shipping	Displays payment and shipping details for each order, including OrderID, OrderDate, PaymentMethod, PaymentDate, PaidAmount, ShippingDate, and ShippingStatus
OrdersWith Coupons	Orders	Customer/ Coupon	Displays orders along with customer details and coupon usage
OrdersWith Payments	Orders	Payment	Provides order and payment information, including OrderID, OrderDate, OrderStatus, PaymentID, PaymentDate, PaidAmount, and PaymentMethod.
OrdersWith Shipping	Orders	Shipping	Displays order shipping details, including OrderID, OrderDate, ShippingID, ShippingDate, ShippingStatus, and Carrier.

ProductInventory Summary	Product	OrderDetails	Provides an overview of product inventory, including ProductID, ProductName, Price, StockQuantity, and the total quantity sold.
TotalSalesPer Customer	Orders	Customer/ OrderDetails	Provides the total amount spent by each customer,

Example of SELECT * on views:

Showing rows 0 - 3 (4 total, Query took 0.0003 seconds.)

SELECT * FROM ActiveOrders;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all

Number of rows: 25

Filter rows: Search this table

Extra options

OrderID

OrderDate

Status

CustomerName

1 2025-03-01 00:00:00 Pending Hayley Williams

2 2025-03-02 00:00:00 Pending Taylor York

3 2025-03-02 00:00:00 Shipped Brian O'Connor

4 2025-03-03 00:00:00 Delivered Zac Farro

Check all

With selected:

Edit

Copy

Delete

Export

Show all

Number of rows: 25

Filter rows: Search this table

Query results operations

Print

Copy to clipboard

Export

Display chart

Create view

Current selection does not contain a unique column. Grid edit, Edit, Copy and Delete features may result in undesired behavior.

Showing rows 0 - 0 (1 total, Query took 0.0011 seconds.)

SELECT * FROM CouponUsageSummary;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all

Number of rows: 25

Filter rows: Search this table

Extra options

OrderID

CouponCode

OrderDate

OrderStatus

CouponID

DiscountPercentage

2 HELLO5 2025-03-02 00:00:00 Pending 4 5.00

9. Explanation and SQL Code for one of your commands to Populate a Tables

	CouponID	Code	DiscountPercentage	ExpiryDate	MinimumOrderValue
	Unique coupon ID	Unique coupon code	Discount percentage (e.g., 15.50 for 15.50%)	Coupon expiry date	Minimum order value required
☐ Edit Copy Delete	1	SPRING20	20.00	2025-06-30 23:59:59	50.00
☐ Edit Copy Delete	2	WINTER15	15.00	2025-12-31 23:59:59	NULL
☐ Edit Copy Delete	3	SUMMER10	10.00	2025-08-31 23:59:59	100.00
☐ Edit Copy Delete	4	HELLO5	5.00	2025-09-30 23:59:59	NULL


```

1 CREATE TABLE `Coupon` (
2   `CouponID` int(11) NOT NULL COMMENT 'Unique coupon ID',
3   `Code` varchar(20) NOT NULL COMMENT 'Unique coupon code',
4   `DiscountPercentage` decimal(5,2) NOT NULL COMMENT 'Discount percentage (e.g., 15.50 for 15.50%)',
5   `ExpiryDate` datetime NOT NULL COMMENT 'Coupon expiry date',
6   `MinimumOrderValue` decimal(10,2) DEFAULT NULL COMMENT 'Minimum order value required'
7 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
8
9 INSERT INTO `Coupon` (`CouponID`, `Code`, `DiscountPercentage`, `ExpiryDate`, `MinimumOrderValue`) VALUES
10 (1, 'SPRING20', 20.00, '2025-06-30 23:59:59', 50.00),
11 (2, 'WINTER15', 15.00, '2025-12-31 23:59:59', NULL),
12 (3, 'SUMMER10', 10.00, '2025-08-31 23:59:59', 100.00),
13 (4, 'HELLO5', 5.00, '2025-09-30 23:59:59', NULL);
14
15 ALTER TABLE `Coupon`
16   ADD PRIMARY KEY (`CouponID`),
17   ADD UNIQUE KEY `Code` (`Code`);
18
19 ALTER TABLE `Coupon`
20   MODIFY `CouponID` int(11) NOT NULL AUTO_INCREMENT COMMENT 'Unique coupon ID', AUTO_INCREMENT=5;
21 COMMIT;

```

One of the tables that I have populated with data is the Coupon table. This stores coupons which will discount a customer's order if applied. Coupons are not necessarily required on all orders.

With an INSERT INTO statement, I have populated four different coupons. They each have a unique coupon ID which serves as the primary key. Additionally, each coupon code must be unique. The coupons are also given an expiry date, from which they cannot longer be redeemed. There is a trigger in the database to enforce this rule.

Some coupons have a minimum-spend requirement. There are triggers that check for orders that try to apply a coupon while not meeting the minimum spend requirement. These orders cannot be inserted into the database, nor updated in a way that would violate this rule.

10. Explanation and example SQL Code for retrieving information from the database

Showing rows 0 - 4 (5 total, Query took 0.0002 seconds)

SELECT o.OrderID, o.CustomerID, o.OrderDate, o.Status, o.TotalAmount, c.Code AS CouponCode, c.DiscountPercentage, c.MinimumOrderValue FROM Orders o LEFT JOIN Coupon c ON o.CouponID = c.CouponID;

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

OrderID	CustomerID	OrderDate	Status	TotalAmount	CouponCode	DiscountPercentage	MinimumOrderValue
1	1	2025-03-01 00:00:00	Pending	3200.00	NULL	NULL	NULL
2	2	2025-03-02 00:00:00	Pending	475.00	HELLO5	5.00	NULL
3	3	2025-03-02 00:00:00	Shipped	1350.00	NULL	NULL	NULL
4	4	2025-03-03 00:00:00	Delivered	800.00	NULL	NULL	NULL
5	5	2025-03-03 00:00:00	Cancelled	0.00	NULL	NULL	NULL

Above is an example of some retrieval SQL code. This code retrieves orders and *joins* coupon details based on coupon ID. The *left join* ensures that orders that do not contain any coupon information are still returned from this query.

✓ Showing rows 0 - 4 (5 total, Query took 0.0002 seconds.)

```
SELECT CustomerID, SUM(TotalAmount) AS TotalSpent FROM Orders GROUP BY CustomerID ORDER BY TotalSpent DESC LIMIT 5;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Extra options

	CustomerID	TotalSpent
☐ Edit Copy Delete	1	3200.00
☐ Edit Copy Delete	3	1350.00
☐ Edit Copy Delete	4	800.00
☐ Edit Copy Delete	2	475.00
☐ Edit Copy Delete	5	0.00

This query orders the top 5 customers in order of how much they have spent. It can be seen from the results of this query that this is not the same as simply retrieving a list of the customers. This can be seen as customers with IDs 3 and 4 have spent more than customer with ID number 2 and therefore appear higher up in the returned data.

11. Explanation and SQL Code for any Security commands

As I do not have the access on macneill to directly *CREATE ROLE*, my Roles and Permissions tables is how I am enforcing security commands.

	role_id	role_name
☐ Edit Copy Delete	3	Admin
☐ Edit Copy Delete	1	Customer
☐ Edit Copy Delete	2	Seller

Roles simply define the roles of a customer, a seller and an admin.

	permission_id	role_id	table_name	can_edit	can_copy	can_delete
☐ Edit Copy Delete	1	3	Orders	1	1	1
☐ Edit Copy Delete	2	3	Coupon	1	1	1
☐ Edit Copy Delete	3	2	Orders	1	1	0
☐ Edit Copy Delete	4	2	Coupon	1	1	0
☐ Edit Copy Delete	5	1	Orders	0	1	0
☐ Edit Copy Delete	6	1	Coupon	0	1	0

Boolean operations in the Permissions table dictate what permissions each role has.

Admins can: edit, copy, and delete from all tables

Sellers can: edit and copy, but cannot delete orders

Customers can: can only copy (view), but cannot edit or delete orders or coupons

The code for these security roles is shown below:

```
11 INSERT INTO `Permissions` (`permission_id`, `role_id`, `table_name`, `can_edit`, `can_copy`, `can_delete`) VALUES
12 (1, 3, 'Orders', 1, 1, 1),
13 (2, 3, 'Coupon', 1, 1, 1),
14 (3, 2, 'Orders', 1, 1, 0),
15 (4, 2, 'Coupon', 1, 1, 0),
16 (5, 1, 'Orders', 0, 1, 0),
17 (6, 1, 'Coupon', 0, 1, 0);
18
19 ALTER TABLE `Permissions`
20   ADD PRIMARY KEY (`permission_id`),
21   ADD KEY `role_id` (`role_id`);
22
23 ALTER TABLE `Permissions`
24   MODIFY `permission_id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=7;
25
26 ALTER TABLE `Permissions`
27   ADD CONSTRAINT `Permissions_ibfk_1` FOREIGN KEY (`role_id`) REFERENCES `Roles` (`role_id`) ON DELETE CASCADE;
28 COMMIT;
```

12. Explanation of and Additional SQL Features of your choice

```
1 DELIMITER $$
2
3 CREATE PROCEDURE NotifyError(IN error_message VARCHAR(255))
4 BEGIN
5     INSERT INTO ErrorLog (ErrorMessage, ErrorTime)
6     VALUES (error_message, CURRENT_TIMESTAMP);
7 END$$
8
9 DELIMITER ;
```

For my advanced SQL feature, I have created a procedure which logs error messages into an ErrorLog table. This has a real-world use for crash reporting. For example an administrator could use the following CALL: “CALL NotifyError('Database connection failed');” and the error message would be stored in the ErrorLo